Dijkstra's Algorithm

Shortest Path in Weighted Graphs | Graph Theory & Algorithms

What is Dijkstra's Algorithm?

Dijkstra's Algorithm, developed by Dutch computer scientist Edsger W. Dijkstra in 1956, is a graph search algorithm that solves the single-source shortest path problem for a graph with non-negative edge weights. It efficiently finds the shortest (minimum cost) path from a starting node to all other nodes in a weighted graph.

The algorithm is widely used in real-world applications such as GPS navigation systems, network routing protocols (like OSPF), and AI pathfinding in games.

Key Properties

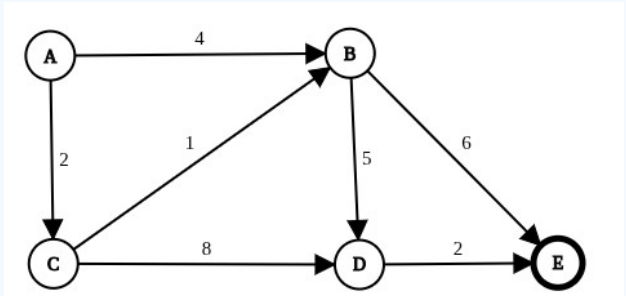
- Works only with non-negative edge weights
- Guarantees the globally optimal (shortest) path
- Uses a greedy approach — always selects the closest unvisited node
- Time complexity: O((V + E) log V) with a priority queue (min-heap)
- Space complexity: O(V) for the distance and visited arrays

How It Works — Step by Step

1. Initialize — Set the distance of the source node to 0 and all other nodes to infinity (∞).
2. Pick the unvisited node with the smallest known distance (start with the source).
3. Update (relax) the distances to each of its unvisited neighbors: if current distance + edge weight < known distance, update it.
4. Mark the current node as visited — it will not be checked again.
5. Repeat steps 2–4 until all nodes have been visited or the target node is reached.

Example Graph

Consider the following directed weighted graph with 5 nodes (A–E) and 7 edges. We want to find the shortest path from node A to all other nodes.



Graph Edges (Directed)

From	To	Weight
A	B	4
A	C	2
B	D	5
C	B	1
C	D	8
D	E	2
B	E	6

Shortest Path Calculation Table (Source: Node A)

The table below traces each step of Dijkstra's algorithm. Bold values indicate the shortest confirmed distance for that node. ∞ means the node has not yet been reached.

Step	Visit Node	Dist A	Dist B	Dist C	Dist D	Dist E
0	Start (A)	0	∞	∞	∞	∞
1	A	0	4	**2**	∞	∞
2	C (dist=2)	0	**3**	2	10	∞
3	B (dist=3)	0	3	2	**8**	9
4	D (dist=8)	0	3	2	8	**9**
5	E (dist=9)	0	3	2	8	9

Final Shortest Distances from A

- A → A : 0
- A → B : 3 (via A → C → B)
- A → C : 2 (via A → C)
- A → D : 8 (via A → C → B → D)
- A → E : 9 (via A → C → B → E)

Limitations

- Does NOT work with negative edge weights — use Bellman-Ford instead.

- Can be slow on very large, dense graphs without an efficient priority queue.
- Finds shortest path by cost, not by number of edges (use BFS for unweighted graphs).